SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

KPIT | APEX LAB

CAN-Monitor, Simulation and Diagnostics Platform

Platform: Raspberry Pi 5 + MCP2515 (SPI CAN Controller) + PCAN USB Interface

1. Introduction

1.1. Purpose

This document defines the requirements, architecture, interfaces, and implementation details of a CAN communication and fault injection platform built using a Raspberry Pi 5, MCP2515 CAN controller, and PCAN USB interface.

The system is intended to:

- Establish CAN bus communication between embedded and host systems.
- Monitor and analyze CAN messages in real time.
- Generate and transmit CAN frames for testing purposes.
- Simulate network conditions and communication faults.
- Provide a platform for validating CAN-based applications and Electronic Control Unit (ECU) behavior.

This document serves as a reference for developers, testers, and stakeholders involved in the design, implementation, and verification of the system.

1.2. Scope

The project focuses on developing a CAN communication and testing platform using Raspberry Pi 5 as the main processing unit and MCP2515 as the external CAN controller connected through the SPI interface.

The scope includes:

- Configuration of MCP2515 CAN controller on Raspberry Pi 5.
- CAN frame transmission and reception.
- Support for Standard (11-bit) and Extended (29-bit) CAN identifiers.
- CAN bus monitoring and logging.
- Fault generation and error simulation.
- Message filtering and acceptance filtering.
- Integration with PCAN USB interface for monitoring and validation.
- Development of user interfaces and diagnostic tools.
- Storage and visualization of CAN traffic.

1.3. Definitions

Acronym	Full Form
ASAM	Association for Standardization of Automation and Measuring Systems
BRS	Bit Rate Switch
CAN	Controller Area Network
CAN FD	Controller Area Network with Flexible Data-Rate
CE	Chip Enable (also Chip Select)
CF	Consecutive Frame
CPOL	Clock Polarity
CPHA	Clock Phase
CRC	Cyclic Redundancy Check
DB9	D-Subminiature 9-pin Connector
DBC	Database CAN (file format)
DID	Data Identifier
DLC	Data Length Code
DTC	Diagnostic Trouble Code
ECU	Electronic Control Unit
EOF	End of Frame
ESI	Error Status Indicator
GPIO	General Purpose Input / Output
GIL	Global Interpreter Lock
IDE	Identifier Extension Bit
IFS	Inter-Frame Space
ISO	International Organization for Standardization
ISO-TP	ISO Transport Protocol (ISO 15765-2)
JSON	JavaScript Object Notation
LIN	Local Interconnect Network
MCU	Microcontroller Unit
MCP2515	Microchip Technology CAN Controller (SPI interface)
MF4	Measurement Data Format version 4 (ASAM standard)
MISO	Master In Slave Out
MOSI	Master Out Slave In
MSB	Most Significant Bit
NFR	Non-Functional Requirement
NRC	Negative Response Code
OTA	Over-The-Air
OVFLW	Overflow / Abort
PCAP	Packet Capture (file format)
PCAN	Peak Controller Area Network (PEAK System USB interface)
PCI	Protocol Control Information
PGN	Parameter Group Number

PID	Parameter Identifier
PTP	Precision Time Protocol (IEEE 1588)
RAM	Random Access Memory
REC	Receive Error Counter
ROM	Read-Only Memory
RPM	Revolutions Per Minute
RTR	Remote Transmission Request
SA	Source Address
SBCs	Single Board Computers
SCLK	Serial Clock
SF	Single Frame
SID	Service Identifier
SPI	Serial Peripheral Interface
SPRM	Suppress Positive Response Message
SPN	Suspect Parameter Number
SRS	Software Requirements Specification
SS	Slave Select
STmin	Separation Time Minimum
TA	Target Address
TEC	Transmit Error Counter
TLS	Transport Layer Security
TPM	Trusted Platform Module
UDS	Unified Diagnostic Services (ISO 14229)
USB	Universal Serial Bus
VIN	Vehicle Identification Number
WebSocket	WebSocket Protocol (RFC 6455)
WT	Wait (ISO-TP Flow Status)

2. Overall description

2.1. System Overview

Hardware needed for this project:

1. Raspberry pi 5
2. MCP2515
3. PCAN USB Converter

2.2. User Classes

- **Developer/Engineer:** Uses diagnostic and simulation features
- **Tester:** Monitors CAN messages and validates behavior

3. Interface Requirements

3.1. SPI Interface (Raspberry Pi ↔ MCP2515)

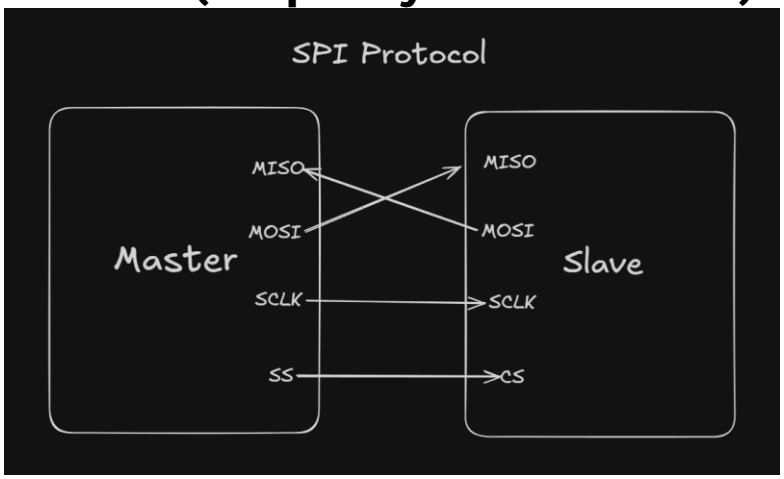

Figure 1 SPI Physical connections.

3.1.1. Working of SPI Protocol

SPI uses two shift registers connected through MOSI and MISO lines. The steps are described below:

1. Slave selection
   The master selects the target slave by activating its CE/SS line.
   CE = LOW - Slave Selected
   CE = HIGH - Slave Deselected
2. Data Loading
   The master loads the data byte into its shift register.
   The slave may also preload data into its shift register if it needs to transmit information back to the master.
3. Clock Generation
   The master generates clock pulses on SCLK.
   Each clock pulse shifts one bit from both shift registers simultaneously.
4. Simultaneous Data Exchange

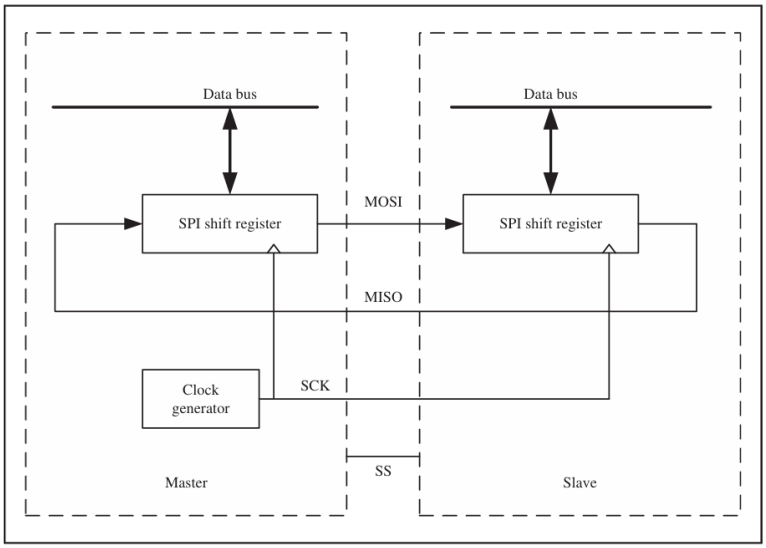

Figure 2 Data exchange in SPI.

For every clock cycle:

- One bit moves from master - slave through MOSI.
- One bit moves from slave - master through MISO.

SPI is therefore a full-duplex protocol, allowing transmission and reception at the same time.

5. Completion of Transfer

After eight clock pulses:

- One byte has been transmitted.
- One byte has been received.
- Communication can continue or terminate by deactivating CE.

Characteristics:

- SPI transfers data one bit at a time synchronized by SCLK.
- MSB is transferred first.
- Communication occurs in groups of 8 bits
- Address and data bytes are transferred sequentially.
- Clock synchronization is provided by SCLK.

3.1.2. SPI Modes

3.1.2.1. CPOL

Defines the idle state of the clock.

CPOL = 0 - Clock remains LOW when idle.

CPOL = 1 - Clock remains HIGH when idle.

SPI Mode	CPOL	CPHA	Data Read (Sample) Edge	Data Change Edge
Mode 0	0	0	Rising Edge	Falling Edge
Mode 1	0	1	Falling Edge	Rising Edge
Mode 2	1	0	Falling Edge	Rising Edge
Mode 3	1	1	Rising Edge	Falling Edge

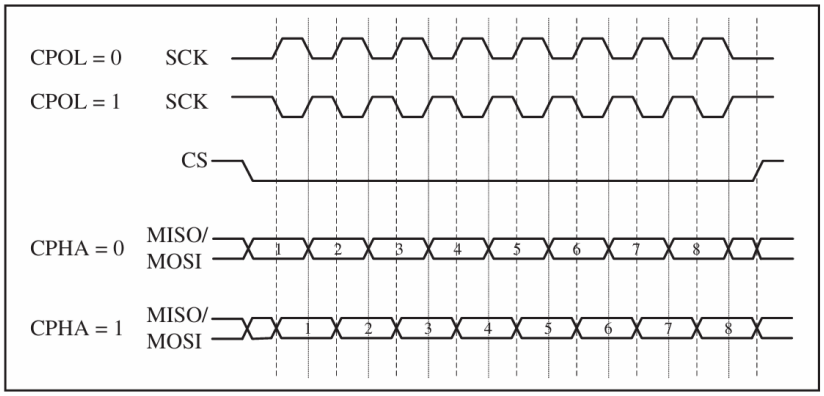

Figure 3 Different SPI modes and their timing.

3.1.2.2. SPI Write Operation

For a single byte write operation:

1. Pull CE LOW to start communication.
2. Send the 8-bit address.
3. The MSB (A7) is set to 1 to indicate a write operation.
4. Send the 8-bit data value.
5. Pull CE HIGH to end communication.

3.1.2.3.SPI Read Operation

For a single-byte read operation:

1. Pull CE LOW.
2. Send the address byte.
3. The MSB (A7) is set to **0** to indicate a read operation.
4. Slave returns the requested data byte.
5. Pull CE HIGH to finish communication.

3.1.2.4. Burst Write

1. Send first address.
2. Send first data byte.
3. Continue sending additional bytes while CE remains LOW.
4. Device automatically increments addresses.

3.1.2.5.Burst Read

1. Send starting address.
2. Device continuously outputs consecutive data bytes.
3. Internal address increments automatically.
4. Communication continues until CE becomes HIGH.

3.2. CAN Interface

3.2.1. Physical Connections

Signals type

1. Differential signal – Value of bit transmitted depends on the difference in between two signals sent:
   For logically “1” -> difference between signal is zero
   For logically “0” -> difference between signal is not zero

   Therefore “0” is dominant and “1” is recessive.

2. Bus

   Two wires (CAN_H and CAN_L).

   The wires should be twisted around each other to cancel out noise. Nodes are connected by drawing wires from this bus for N nodes wire requirement is 2(N + 1).

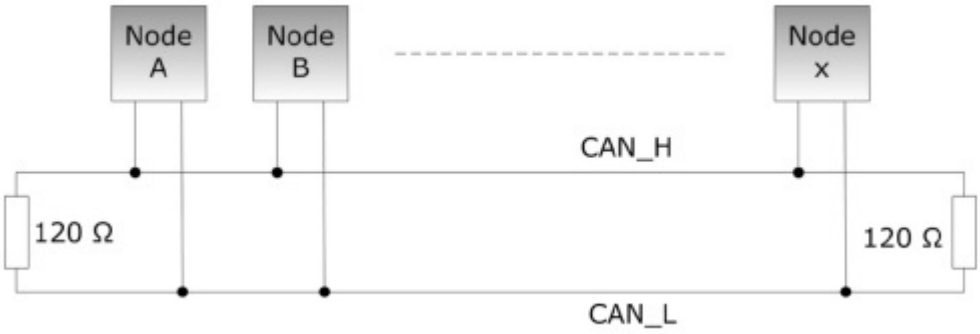

Figure 4 CAN Bus Connection.

3. 120 Ohms resistance

   A CAN bus cable behaves as a transmission line and has a property called characteristic impedance Z_0 which is the ratio of voltage to current for a travelling wave on the cable and is determined by the cable's distributed inductance and capacitance:

$$Z_0 = \sqrt{\frac{L}{C}}$$

   For CAN twisted-pair cables, $Z_0 \approx 120\Omega$. To prevent signal reflections, the load impedance must match the cable impedance. The amount of reflection is given by:

$$\Gamma = \frac{R_L - Z_0}{R_L + Z_0}$$

   Where R_L is the load impedance.

   1. Open circuit ($R_L = \infty$): $\Gamma = 1 \rightarrow$ reflection with the same polarity.
   2. Short circuit ($R_L = 0$): $\Gamma = -1 \rightarrow 100\%$ reflection with inverted polarity.
   3. Matched load ($R_L = 120\,\Omega$): $\Gamma = 0 \rightarrow$ no reflection; all signal energy is absorbed by the load.

   In a CAN network, 120 Ω termination resistors are placed at both physical ends of the bus so that a travelling wave reaching either end sees a 120 Ω load, resulting in $\Gamma = 0$ and eliminating reflections. Although a multimeter measures about 60 Ω across CAN_H and CAN_L (because the two 120 Ω terminators are in parallel), the travelling wave still sees 120 Ω at each end, which is what provides proper impedance matching.

3.3. ECU structure

Consist of three main parts MCU, CAN controller, CAN transmitter

3.3.1. Microcontroller Unit

Executes application software. Reads sensors and controls actuators. Sends and receives CAN data through the CAN controller.

3.3.2. CAN Controller

Implements CAN protocol functions.

Performs:

- Frame generation
- Arbitration
- CRC generation/checking
- Error detection
- Message filtering

3.3.3. CAN Transceiver

Converts logic signals into differential CAN signals.

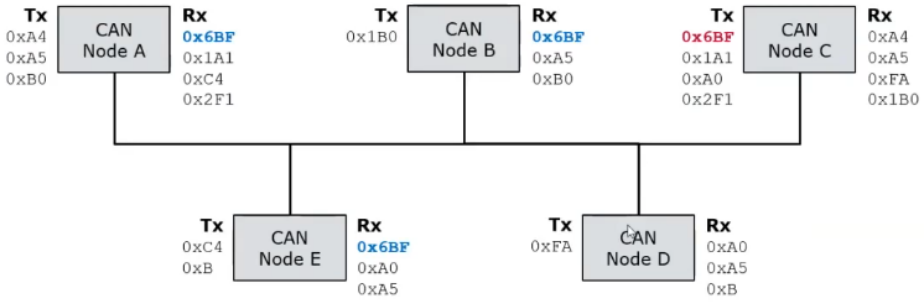

Figure 5 CAN Data transmission, reception and filtering.

Drives CAN_H and CAN_L. Receives signals from the CAN bus.

3.4. CAN Types

3.4.1. Based on Identifier Size

Standard CAN (Normal CAN) Uses an 11-bit Message Identifier (ID):

Supports 0–8 bytes of data.

Extended CAN Uses a 29-bit Message Identifier (ID):

Supports 0–8 bytes of data.

3.4.2. Based on Frame Sent

Data Frame Used to transmit actual application data. Most frequently used CAN frames. Contains both Message ID and Data Field.

Remote Frame Used to request data from another ECU. Does not contain any data bytes. RTR (Remote Transmission Request) bit is set.

3.5. CAN FD

CAN FD is an enhanced version of Classical CAN introduced to increase bandwidth and payload capacity. It supports payloads of up to **64 bytes** and

allows a higher bit rate during the data phase while maintaining the standard CAN arbitration mechanism.

Message Signal in Detail

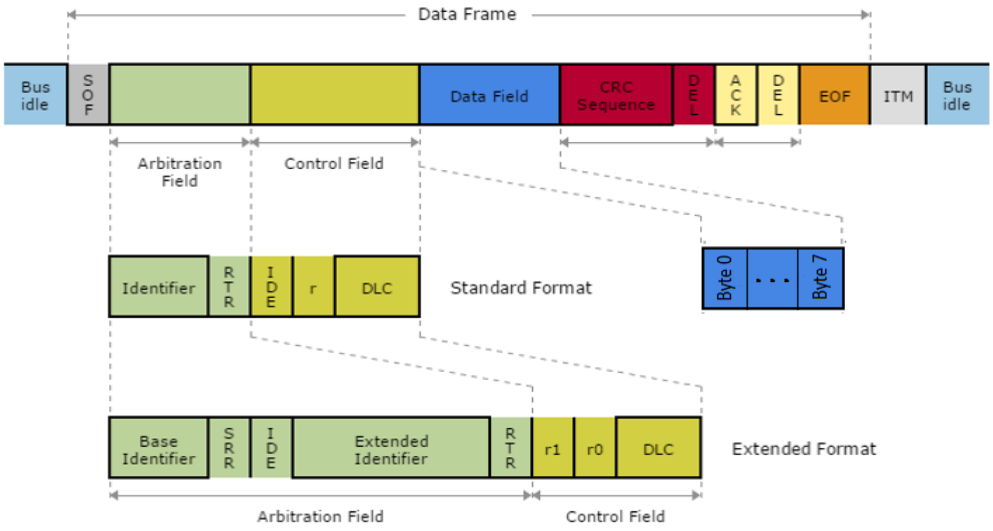

Figure 6 CAN message structure.

1. **SOF (Start of Frame) -** Marks the beginning of a new CAN frame. Consists of 1 dominant bit (0).
2. **Identifier Field -** Contains the Message ID. Standard CAN: 11 bits, Extended CAN: 29 bits Determines message priority during arbitration. Sent MSB (Most Significant Bit) first.
3. **RTR (Remote Transmission Request) -**
    a. Data Frame (RTR = 0 (Dominant)) - Indicates that the frame contains actual data.
    b. Remote Frame (RTR = 1 (Recessive)) - Requests data from another node. No data field is transmitted.
4. **IDE (Identifier Extension Bit) -** Determines identifier format.
    a. When IDE = 0 -> Standard CAN. Uses 11-bit identifier.
    b. When IDE = 1 -> Extended CAN. Uses 29-bit identifier.

5. **DLC (Data Length Code)** - 4-bit field. Specifies the number of data bytes.

Frame	DLC	Number of Data Bytes
CAN 2.0 and CAN FD	0	0
	1	1
	2	2
	3	3
	4	4
	5	5
	6	6
	7	7
	8	8
CAN 2.0	9-15	8
CAN FD	9	12
	10	16
	11	20
	12	24
	13	32
	14	48
	15	64

6. **Data Field** - Actual payload information. Contains application data.
    a. Classical CAN - 0 to 8 bytes
    b. CAN FD - 0 to 64 bytes
7. **CRC Field (Cyclic Redundancy Check) -** Used for error detection.
    a. Transmission Process - Transmitter calculates CRC from all previous bits. CRC value is appended to the frame. Receivers perform the same CRC calculation. Calculated CRC is compared with received CRC.
    b. Result:
    c. Match - Frame valid.
    d. Mismatch - Error Frame generated.
8. **ACK (Acknowledgement Field) -** Used to confirm successful reception.
    a. If frame received correctly: Receiver writes dominant bit (0)
    b. If nobody acknowledges: Bus remains recessive (1) and transmitter retries later.
9. **ACK Delimiter:** One recessive bit. Separates ACK from EOF.
10. **EOF (End of Frame):** Consists of 7 recessive bits (1111111). Marks the end of transmission. Indicates frame completion.

3.5.1. Arbitration steps

- Bus is idle (all nodes see recessive state).
- Multiple nodes start transmitting at the same time.
- Each node transmits its Message ID bit-by-bit.
- While transmitting, each node also monitors the bus.
- If a node transmits 1 (recessive) but reads 0 (dominant):
- It realizes another node has a higher-priority message.

- It immediately stops transmitting.
- The node that never detects a mismatch wins arbitration.
- Winning node continues transmitting the complete frame.
- Losing nodes retry transmission when the bus becomes idle again.

3.5.2. How CAN FD is different

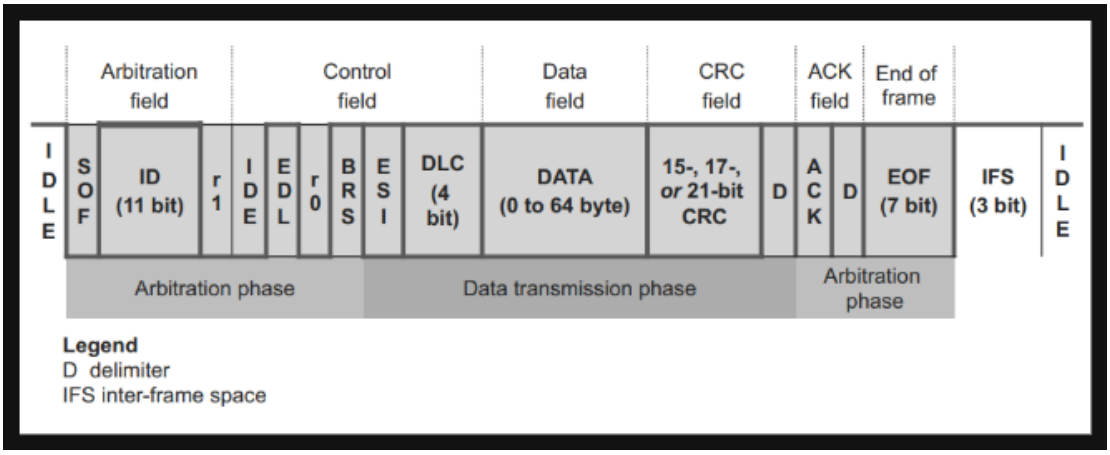

Figure 7 CAN FD message structure.

- CAN FD starts transmission at the nominal bit rate (e.g., 500 kbps) for the SOF, Identifier, and Arbitration fields.
- If the BRS (Bit Rate Switch) bit is set, all nodes switch to a higher data bit rate (e.g., 2–8 Mbps).
- The Data Field and CRC Sequence are transmitted at this higher speed, reducing transmission time.
- After the CRC field, all nodes switch back to the nominal bit rate.
- The ACK, EOF, and Inter-Frame Space (IFS) are transmitted at the nominal bit rate to ensure reliable acknowledgment and bus synchronization.

3.5.3. Bit Stuffing

Bit stuffing is an error detection and synchronization mechanism used in CAN communication. After **five** consecutive bits of the same value, the transmitter automatically inserts an additional bit of the opposite value. This inserted bit is called a stuff bit.

- Bit Stuffing Transmission.
  - The transmitter monitors the bit stream being sent.
  - After 5 consecutive identical bits (0s or 1s), it automatically inserts a bit of the opposite value.
  - The inserted bit is called a stuff bit.
  - Bit stuffing is applied from SOF to CRC Sequence.
- Bit Stuffing Reception.
  - Receiver monitors incoming bits.
  - When it detects 5 identical bits followed by an opposite bit, it recognizes the opposite bit as a stuff bit.
  - The stuff bit is removed before processing the message.

- The original data is reconstructed.

3.5.4. Error Frame

- An Error Frame is transmitted when a CAN node detects an error in a received or transmitted message.
- Inform all nodes that the current frame is corrupted.
- Cause all nodes to discard the erroneous frame.
- Force retransmission of the message.

Operation

- A node detects an error.
- It immediately transmits an Error Flag.
- Other nodes detect the Error Flag and stop processing the current frame.
- An Error Delimiter follows.
- The bus returns to idle state.
- The original transmitter retries sending the frame.

3.5.4.1. Types of CAN Errors

- Bit Error: A node transmits a bit but reads a different value from the bus.
- Stuff Error: More than 5 consecutive identical bits are received without a valid stuff bit.
- CRC Error: Calculated CRC does not match the received CRC.
- Form Error: A fixed-format field contains an invalid bit value.
- ACK Error: Transmitter does not receive a dominant bit in the ACK slot.

3.5.4.2. CAN Error States

CAN uses **Transmit Error Counter (TEC)** and **Receive Error Counter (REC)** to monitor node health. Based on these counters, a node can be in one of three states:

- Error Active State
  - Condition: TEC < 128 & REC < 12.
- Error Passive State
  - Condition: TEC ≥ 128 | REC ≥ 128
- Bus-Off State
  - Condition: TEC ≥ 256

3.6. PCAN USB Interface

3.6.1. Physical Pinout (D-Sub 9-pin)

The adapter exposes the CAN interface through a standard male **9-pin D-Sub (DB9)**

Pin Number	Signal Name	Description
1	NC / +5V	Not connected by default (Optional +5V output via internal solder jumper)
2	CAN_L	CAN Low signal (dominant low)
3	GND	Ground
4	NC	Not connected
5	NC	Not connected
6	GND	Ground (tied internally to pin 3)
7	CAN_H	CAN High signal (dominant high)
8	NC	Not connected
9	NC / +5V	Not connected by default (Optional +5V output via internal solder jumper)

3.7. ISO-TP Interface (ISO 15765-2)

The ISO-TP (ISO 15765-2) protocol is a standardized transport layer protocol built on top of Controller Area Network (CAN). Standard CAN is limited to a maximum data payload of **8 bytes** per frame.

ISO-TP solves this limitation by acting as a segmentation and reassembly layer, allowing the transmission of payloads up to **4,095 bytes** (and up to **4 GB** in updated revisions like ISO 15765-2:2016 over CAN FD).

3.7.1. Frame Types

Four distinct frame types, identified by the first 4 bits of PCI (Byte 0)

a) **Single Frame (SF)**
   i. If the data payload is less than 7 bytes, it is transmitted in a SF.
   ii. Byte 0 (PCI): Lower 4 bits represent the Data Length (DL).
   iii. Bytes 1–7: The actual payload.

b) **First Frame (FF)**
   i. When the payload exceeds 7 bytes, the transmitter issues an FF to alert the receiver.
   ii. Byte 0–1 (PCI): The lower 4 bits of Byte 0 combined with Byte 1 form a 12-bit Data Length (DL) field, allowing for sizes up to 4,095 (2 ^ 12 - 1) bytes.
   iii. Bytes 2–7: The first 6 bytes of the data payload.

c) **Flow Control (FC)**

i. After receiving an FF, the receiver must tell the transmitter how it wants to receive the remaining data to avoid buffer overflows.
ii. Byte 0: **Flow Status (FS).**
iii. 0 = Continue to Send (CTS)
iv. 1 = Wait (WT), 2 = Overflow/Abort (OVFLW).
v. Byte 1: **Block Size (BS).** The number of Consecutive Frames the transmitter is allowed to send before waiting for the next Flow Control frame. 0 means sending all remaining frames without another pause.
vi. Byte 2: **Separation Time.** The minimum time the transmitter must wait between sending Consecutive Frames.

d) **Consecutive Frame (CF)**
i. These frames contain the rest of the payload data.
ii. Byte 0 (PCI): The lower 4 bits represent a Sequence Number (SN). The SN starts at 1 since the First Frame acts as index 0, increments up to 15 (0xF), and then wraps back around 0.
iii. Bytes 1–7: The remaining data chunks.

3.7.2. Communication Flow

a) Sender transmits First Frame and tells the receiver exactly how many bytes to expect.
b) The receiver checks its buffer availability and replies with an FC frame specifying the allowed Block Size and STmin (inter-frame delay).
c) The sender sends bursts of data frames matching the limits set by the receiver.
d) If the total data size exceeds the Block Size (BS), the sender stops after the designated number of frames, waits for another Flow Control frame from the receiver, and resumes sending CFs. This repeats until all bytes are transferred.

3.7.3. Addressing Formats

Addressing formats define how the network layer maps the sender (source) and receiver (target) addresses onto the underlying CAN frame structure. The protocol uses either the CAN Identifier (11-bit or 29-bit), the first byte of the data payload, or a combination of both.
Choosing an addressing format changes how many bytes remain available for actual diagnostic payload inside the CAN frame.

a) **Normal Addressing:** The target and source addresses are encoded completely within the standard 11-bit or extended 29-bit CAN Identifier.
b) **Normal Fixed Addressing:** The 29-bit CAN ID is strictly segmented into dedicated bitfields: Priority, Source Address (SA), and Target Address (TA).
c) **Extended Addressing:** The CAN ID defines the message type, but the very first byte of the CAN data field (Byte 0) is utilized as the Target

Address (TA). This reduces a Single Frame's maximum data payload from 7 bytes down to 6 bytes.

3.7.4. Network Later Timeouts

Transmission Timeouts

a) **N_As (Sender side):** The maximum time allowed for the sender to physically transmit an ISO-TP frame (FF, CF, or SF) over the CAN bus. If the bus is heavily loaded with higher-priority IDs, the frame might get delayed in the CAN controller's transmit buffer. If it exceeds N_As, the transmission aborts.

b) **N_Ar (Receiver side):** The maximum time allowed for the receiver to physically transmit a Flow Control (FC) frame.

Flow Control Handshake Timeouts

a) **N_Bs (Sender side):** The maximum time the sender will wait for a Flow Control (FC) frame from the receiver after it has finished sending a First Frame (FF).

b) **N_Br (Receiver side):** The internal delay time for the receiver to prepare and trigger the transmission of the Flow Control frame.

Consecutive Frame Transfer Timeouts

a) **N_Cs (Sender side):** The internal preparation time for the sender before it transmits a Consecutive Frame (CF). This is highly dependent on the Frame Separation Time parameter dictated by the receiver's FC frame.

b) **N_Cr (Receiver side):** The maximum time the receiver will wait for the arrival of the next Consecutive Frame (CF).

3.8. UDS Interface (ISO 14229)

3.8.1. Request and Response Communication

UDS follows a strict Client-Server (or Tester-ECU) relationship. The diagnostic tester sends a request, and the target ECU responds. With every UDS request starts with a 1-byte Service Identifier (SID).

- Requests: The SID byte determines the action (e.g., 0x22 for reading data).
- Positive Responses: This tells the tester that the command was successfully understood and executed.
- Negative Responses: If something goes wrong, the ECU responds with a Negative Response

3.8.2. Message Structure

Every diagnostic request sent by a client (tester) to an ECU follows a structural layout based on whether the service uses a Sub-function or direct Data Identifiers (DIDs).

Requests

Requests with a Sub-Function

Services that change modes or trigger states (like 0x10 Session Control, 0x11 ECU Reset, 0x28 Communication Control):

- **Byte 0 - SID:** The specific service requested (e.g., 0x10).
- **Byte 1 - Sub-Function:** Details on how to execute the service (e.g., 0x03 for Extended Session).
- **Byte 2 to N - Parameters:** Extra configuration data if required by that sub-function.

Sub Function Byte

Bit 7: Suppress Positive Response Message (SPRM):

- 0: The ECU must send a positive response when done.
- 1: The ECU must stay silent if the command succeeds and it will only respond if it fails.

Bits 6 to 0: Sub-Function Value. The command parameter (values 0x00 to 0x7F).

3.8.3. Requests without a Sub-Function

Services that read/write parameters or memory directly (like 0x22 Read Data, 0x2E Write Data) do not have sub-functions. Instead, they pass raw target identifiers immediately.

- **Byte 0 - SID:** 0x22 for Read or 0x2E for Write.
- **Byte 1 & 2 - DID:** 2 bytes unique address representing a specific signal or variable (e.g., 0x210A for Battery Voltage).
- **Byte 3 to N:** For a Read request, this is empty. For a Write request, this contains the raw hex bytes to be written into that DID.

3.8.4. Positive Response Message Structure

When a request clears all validation checks and executes successfully, the ECU builds a Positive Response.

Sub-Function Service Responses

- Byte 0: SID + 0x40
- Byte 1: Echoed Sub-Function
- Byte 2: N Response Data

Sub-Function Service Responses

- Byte 0: SID + 0x40
- Byte 1 & 2: Echoed DID
- Byte 3 – N: Actual Data Content

Negative Response Message Structure

If a service fails evaluation at the application level, the ECU completely drops the positive response template and outputs a rigid, mandatory 3-byte Negative Response:

- Byte 0: Negative Response SID (0x7F)
- Byte 1: Rejected SID, the exact SID of the command that failed.
- Byte 2: NRC, the standard 1-byte failure cause code (e.g., 0x33 for Security Access Denied, 0x13 for Incorrect Message Length).

4. Functional Requirements:

This section outlines what the system is expected to do across its four operational areas: CAN Monitoring, Simulation, Diagnostics, and Logging. Rather than prescribing implementation details, each requirement focuses on observable behavior what goes in, what happens to it, and what comes out. Every requirement follows a consistent layout covering an ID, title, inputs, processing steps, outputs, and a priority level.

4.1. CAN Monitoring

4.1.1. CAN Communication Support (500 kbps)

Title: Standard CAN Communication Support

Description: The system shall support Controller Area Network (CAN) communication at a baud rate of 500 kbps, enabling reliable message exchange between the Raspberry Pi 5 and external ECUs on the vehicle network.

Inputs:

- CAN bus signals from external ECUs.
- Configuration parameters (bitrate set to 500 kbps).
- CAN transceiver hardware (MCP2515 or equivalent).

Processing:

- Initialize CAN interface on Raspberry Pi 5 using SocketCAN (ip link set can0 up type can bitrate 500000).
- Configure bitrate to exactly 500 kbps with correct sample point (87.5%).
- Continuously listen on CAN bus for incoming frames using non-blocking read loop.
- Transmit CAN frames on request from upper application layers.
- Monitor interface state and reinitialize on error

Outputs:

- Successfully transmitted CAN frames on the bus.
- Received CAN messages passed to decoding layer .
- Interface status (up / down / error) logged to system log

Priority: High.

4.1.2. CAN Message Decoding Using DBC Files

Title: DBC-Based CAN Message Decoding

Description: The system shall decode raw CAN frames into human-readable engineering signal values using industry-standard DBC (Database CAN) files, applying the correct scaling, offset, and unit conversion for each signal.

Inputs:

- Raw CAN frames (frame ID + payload bytes) from CAN interface.
- DBC file(s) loaded by the user or pushed via OTA .
- Signal definitions within DBC: message ID, signal name, start bit, length, byte order, scale, offset, min, max, unit.

Processing:

- Parse and load DBC file into memory at system startup
- For each received CAN frame, look up message ID in DBC message table
- Extract signal bit fields from payload using defined start bit, length, and byte order (Intel / Motorola)
- Apply scaling formula: Physical Value = (Raw Value × Scale) + Offset
- Validate decoded value against DBC min/max range
- Map decoded signals to named signal objects with value and unit.

Outputs:

- Decoded signal values with engineering units (e.g. Engine RPM = 3500 rpm, Vehicle Speed = 80 km/h).
- Named signal objects passed to monitoring interface (FR7) and log system (FR5).
- Out-of-range signal warnings passed to validation layer (FR38).

Priority: High.

4.1.3. CAN FD Communication Support (Up to 64 Bytes Payload)

Title: CAN Flexible Data-Rate (FD) Communication Support.

Description: The system shall support CAN FD protocol, enabling data frames with payloads of up to 64 bytes and faster data-phase bitrates, while remaining backward compatible with classical CAN frames on the same bus.

Inputs:

- CAN FD frames from external ECUs with up to 64-byte payload
- Configuration parameters (nominal bitrate: 500 kbps, data-phase bitrate: up to 2 Mbps).
- CAN FD be capable transceiver (MCP251xFD or equivalent).

Processing:

- Initialize CAN FD interface using SocketCAN with FD mode enabled (ip link set can0 up type can bitrate 500000 dbitrate 2000000 fd on).
- Configure nominal bitrate at 500 kbps and data-phase bitrate up to 2 Mbps.
- Detect frame type on reception (classical CAN vs CAN FD) and process accordingly.
- Allocate extended 64-byte frame buffer for CAN FD frames.
- Transmit CAN FD frames with correct BRS (Bit Rate Switch) and ESI (Error Status Indicator) flags set.

Outputs:

- Successfully received and transmitted CAN FD frames with up to 64-byte payload.
- Frame type tag (CAN / CAN FD) attached to every captured frame.
- CAN FD frame data be passed to decoding layer .

Priority: High.

4.2. Simulation

4.2.1. ECU Message Simulation (RPM, Speed, Temperature)

Title: ECU Signal Simulation.

Description: The system shall simulate realistic ECU CAN messages for key vehicle signals including Engine RPM, Vehicle Speed, and Engine Coolant Temperature, transmitting them on the CAN bus at correct cycle times as defined in the loaded DBC file.

Inputs:

- DBC file defining simulated message IDs, signal encoding, and cycle times.
- Simulation parameters: signal names, value ranges, waveform type (static / ramp / sine / random).
- User-configured simulation scenario (e.g. engine idle, acceleration, thermal ramp).

Processing:

- Load message definitions and cycle times from DBC for signals: Engine RPM, Vehicle Speed, Coolant Temperature.
- Generate signal values by selected waveform:
  - Static: fixed value throughout simulation.
  - Ramp: linearly increase/decrease between min and max over defined duration.
  - Sine: sinusoidal oscillation between min and max at defined frequency.
  - Random: uniformly random value within min/max bounds per cycle.
- Encode signal values into CAN payload bytes using DBC bit layout and scaling (reverse of FR3 decoding).

- Transmit encoded CAN frame on bus at DBC-specified cycle time using periodic timer.

Outputs:

- CAN frames containing simulated RPM, Speed, and Temperature signals transmitted at correct cycle times.
- Simulation status display in monitoring interface showing current simulated values.
- All simulated frames captured in session log tagged as "simulated".

Priority: High

4.2.2. Real-Time Monitoring Interface

Title: Real-Time CAN Monitoring Interface

Description: The system shall provide a real-time graphical monitoring interface displaying live decoded CAN signal values, bus statistics, ECU status, and active alerts, updated continuously as data arrives from the CAN bus.

Inputs:

- Live decoded signal values from DBC decoder.
- Bus load percentage from performance monitor.
- ECU heartbeat status from heartbeat monitor.
- Active fault alerts from fault management layer.
- Raw frame stream for frame-level view.

Processing:

- Receive decoded signal updates and push to UI render pipeline.
- Refresh signal value display at minimum 10 Hz update rate.
- Render live time-series chart for user-selected signals (scrolling 30-second window).
- Display bus load gauge updated every 100 ms.
- Show ECU status table with heartbeat indicator per ECU (green / yellow / red).
- Display active alert banner for any fault condition.
- Provide searchable signal list allowing users to pin signals to dashboard.

Outputs:

Priority: High.

4.3. Diagnostics

4.3.1. Communication Error Handling with Retries

Title: CAN Communication Error Handling and Retry Mechanism

Description: The system shall detect CAN communication errors including transmission failures, bus-off states, and timeout conditions, and automatically attempt recovery through a configurable retry mechanism without requiring manual intervention.

Inputs:

- CAN controller error status register (TEC, REC counters).
- Transmission acknowledgement failure flag from CAN controller.

- Configurable retry parameters: maximum retry count (default: 3), retry interval (default: 10 ms), backoff multiplier (default: 2×).

Processing:

- On each CAN frame transmission attempt, check for acknowledgement error (no ACK received).
- If transmission fails, wait for retry interval and retransmit frame.
- Apply exponential backoff: retry interval doubles on each consecutive failure (10 ms → 20 ms → 40 ms).
- After maximum retry count exceeded, discard frame and log transmission failure event.
- Continuously monitor TEC and REC error counters.
- On Bus-Off state detected: log event, wait 128 × 11 recessive bits recovery period, reinitialize CAN interface, resume normal operation.
- On repeated Bus-Off within 60 seconds: raise critical alert and halt transmissions pending user acknowledgement.

Outputs:

- Successfully retransmitted frames delivered on bus within retry window.
- Transmission failure log entry: frame ID, payload, timestamp, retry count, failure reason.
- Bus-Off recovery event logged with timestamp and recovery duration.
- Critical alert raised in monitoring interface if recovery fails.

Priority: High

4.3.2. UDS Diagnostics over ISO-TP

Title: Unified Diagnostic Services (UDS) over ISO-TP

Description: The system shall implement the UDS (ISO 14229) diagnostic protocol transported over ISO 15765-2 (ISO-TP), enabling reading of Diagnostic Trouble Codes (DTCs), ECU identification, and diagnostic session control.

Inputs:

- UDS diagnostic request commands from user or test script (service ID + sub-function + parameters).
- Target ECU CAN ID (request ID and response ID).
- ISO-TP configuration: addressing mode (normal / extended), block size, separation time.

Processing:

- Segment outgoing UDS request into ISO-TP frames (Single Frame, First Frame, Consecutive Frames) based on payload length.
- Transmit ISO-TP frames on CAN bus to target ECU request ID.
- Receive and reassemble ISO-TP response frames from ECU response ID.
- Manage ISO-TP flow control (send Flow Control frame, respect separation time ST from ECU).

- Parse reassembled UDS response: extract service response ID, response code, and data payload.
- Decode UDS response data (e.g. DTC list, ECU serial number, live data PIDs).

Outputs:

- Decoded UDS diagnostic response displayed in diagnostic interface.
- DTC list with fault code, description, and status byte.
- ECU identification data (VIN, ECU hardware/software version).
- UDS transaction log (request + response + timestamp) written to session log.

Priority: High

4.4. Logging

4.4.1. CAN Traffic Logging to Persistent Storage

Title: Persistent CAN Traffic Logging

Description: The system shall continuously log all received and transmitted CAN frames, along with their decoded signal values, to persistent storage in a structured file format that survives power cycles.

Inputs:

- Raw CAN frames (frame ID, DLC, payload, timestamp) from CAN interface.
- Decoded signal values from DBC decoder.
- User configuration: log file path, max file size, rotation policy.

Processing:

- Open log file at session starts with timestamped filename.
- Write each frame entry: absolute timestamp (µs), channel ID, frame ID (hex), DLC, raw payload (hex), decoded signal values.
- Buffer writes in memory (ring buffer, 1000 frames) and flush to disk every 500 ms to reduce I/O overhead.
- Rotate log file when size exceeds configured limit (default: 100 MB), starting new file automatically.
- On shutdown or crash, flush all buffered frames to disk before closing file.
- Apply encryption to log file if FR11 is enabled.

Outputs:

- Structured log files written to configure persistent storage path (SD card / SSD).
- Log file format: internal binary or ASC/MF4 depending on configuration.
- Log session index file listing all session files with start time, end time, frame count.

Priority: High

5. Non-Functional Requirements

5.1. Real Time Performance:(Latency less than 100ms)

As CAN messages are transmitted at 500 kbps. If systems take too long to process a message, you lose data or display wrong readings. When the frame arrives to mcp2515, through decoding, filtering, rendering etc. on the dashboard this whole should be done in 100ms. This latency covers the time for SocketCAN read, Python-can library decoding time and adding data in csv files. Also, messages like errors should prioritized over background logging tasks.

5.2. Modular Python Architecture

CodeBase of Project is needed to split into different modules in which one is for CAN reading, for dashboard, for logging, for filtering, message decoding, etc. So that testing for one component or feature shouldn't be needed to run whole code and different components can be worked by different team members.

5.3. Deployable on Raspberry PI

We have to design the system such that it is compatible with rpi5 as the raspberry pi is arm based SBCs. So, system software needed to be made such that all the libraries like python-can, flask, Matplotilb must be ARM compatible packages. Also, it should not be heavy as RPI may have limited resources like here we used 8GB Ram and 16GB ROM also computational power needed to count.

5.4. Scalability for Additional Signals

The System is to be designed such that it should not to be work with only one parameter like speed as in future we can have changes in automative cars like added gear position sensor or something like to measure battery voltage therefore the code is needed to flexible we should to be able to add new parameter according to demand and need. It needed to be designed such that adding new CAN ID/Signal doesn't require restructuring the whole codebase.
Also, the Code can be made such that another CAN signal also gets worked with it like using one more MCP2515 so that two CAN can be used at a time through a single code file.

5.5. Reliability with Retry Mechanisms

The system needs to implement Retry and Recovery mechanisms at all hardware interface boundaries. like CAN buses drop frames, the MCP 2515 can encounter SPI errors, and USB connections can glitch etc. in those cases system should not throw any random error it should to return messages like retry failed SPI reads, Reconnect to PCAN-US if disconnects etc.

5.6. Data Integrity

There should not be any loss of CAN messages or corruption during acquisition, processing, or logging. All logged data shall be written to prevent partial records in csv/json files.

5.7. Fault Tolerance

The system should be able to detect and handle hardware faults, including SPI communication errors from the MCP2515, USB disconnection of the PCAN-USB interface, and CAN bus-off states. System should resume normal operation within 5 sec of the fault resolutions.

5.8. Logging and Traceability

The system shall maintain timestamped logs of all CAN traffic, system events, errors, and operator actions. Log files shall follow a structured and consistent format to enable post processing and help in analysis. We need to use Log Rotation as we have low space in RPI5's SD card.

5.9. Security

If remote monitoring or network access is enabled, the system shall restrict dashboard access to authorized users through authentication mechanisms. All data transmitted over the network needs to be encrypted using TLS. Unused network ports shall be disabled by default to minimize the attack surface.

5.10. Resource Efficiency

The system should operate with the RPI5's hardware constraints without causing thermal throttling or memory exhaustion under operating condition so to get the most efficient output. Like RAM consumption may be less than 512mb, CAN monitoring shall not exceed 60% on any single core.

5.11. Configuration Flexibility

We should be able to change how the system behaves without touching the code or restarting the system. Everything that can be "tuned"; speeds, paths, intervals, filters should live in one config file, and changes to that file should take effect immediately while the system is still running.

5.12. Observability and Diagnostics

The system exposes internal health metrics including CAN message receive rate, buffer utilization, CPU and memory usage, uptime, error frame counts etc. They can be accessed by Dashboard.

6.

7. Hardware Constrains

7.1. Raspberry Pi 5

- Limited CPU cores (quad-core ARM Cortex-A76) running Flask + Matplotlib + python-can simultaneously competes for CPU.
- RAM is limited (4–8 GB); large log files or high-frequency buffers can cause memory pressure.
- No native CAN controller: you depend entirely on SPI (MCP2515) or USB (PCAN) for CAN access.
- SD card I/O is slow; writing high-frequency CSV logs may create bottlenecks.
- Power draw increases under heavy load; unstable power supply causes crashes.

7.2. MCP2515 CAN

- SPI communication adds latency compared to native CAN controllers (each frame requires SPI transaction overhead)
- Maximum SPI clock frequency limits throughput at high CAN bus loads (1000+ frames/sec), the SPI bridge can become a bottleneck
- Only one CAN channel cannot monitor two buses simultaneously without additional hardware.
- Interrupt handling must be configured correctly, or frames will be missed under heavy traffic.
- Temperature sensitivity: SPI timing can drift at extremes.

7.3. PCAN-USB Interface

- Dependent on USB 2.0 bandwidth and latency (~1 ms USB polling intervals).
- Requires correct kernel drivers (pcan module) on Linux; must be compiled or installed for Pi OS.
- USB bus contention: if other USB devices are active, latency increases.
- Limited to one CAN channel per device.

7.4. Power Supply

- Raspberry Pi 5 requires a stable 5V/5A supply (USB-C PD); insufficient current causes throttling or resets.
- MCP2515 runs on 3.3V from the Pi's GPIO rail; excessive SPI activity can slightly stress this rail.
- PCAN-USB draws power from USB combined with Wi-Fi dongle or other peripherals; total current budget must be planned.
- In automotive environments, supply voltage can be noisy; filtering/regulation is needed.

7.5. Operating Environment

- Raspberry Pi 5 is rated for 0–50°C; automotive environments can exceed this.
- Vibration in a vehicle can loosen SPI wiring or USB connectors.
- Humidity and dust exposure require enclosure protection if deployed in-vehicles.
- No hardware watchdog enabled by default — a software hang won't auto-recover without configuration.

7.6. Software / OS

- SocketCAN (vcan, can0) must be properly configured at boot via systemd or rc.local; this is not automatic.
- Python's GIL (Global Interpreter Lock) limits true parallelism; CPU-bound tasks (Matplotlib rendering) can block CAN reading.
- Real-time Linux scheduling is not enabled by default; critical threads may be preempted by OS tasks
- Flask's development server is single-threaded; under concurrent dashboard access, it may lag.

7.7. Network

- Wi-Fi introduces variable latency (10–100+ ms) which conflicts with NFR1 if the dashboard is remotely hosted
- Raspberry Pi's onboard Wi-Fi antenna has limited range and throughput
- If streaming live CAN data over HTTP/WebSocket, bandwidth must be budgeted (at 500 kbps CAN, decoded JSON output can easily reach several MB/min)
- Network disconnections require the dashboard to gracefully degrade (show cached data, not crash)

8. Conclusions

This SRS defines the interface requirements and functional scope of a robust CAN monitoring, simulation, and diagnostic platform using Raspberry Pi 5. The system integrates low-level hardware communication (SPI, CAN) with higher-level automotive protocols (ISO-TP, UDS), enabling flexible and extensible automotive network analysis